Day 1

Web Application Architecture

1. Web Application Architecture

a) Layers

b) Tiers

---

2. Layers

- Application is built in layers.
- Web Application comprises of 3 layers.

a) Database Layer

b) Business Layer

c) Presentation Layer

Database Layer

- Developer designs database.
- Writes the queries for interacting with database.
- Writes procedures with queries etc.

Business Layer

- Developer writes logic to handle communication between user and database.
- It represents application center with middleware.

Presentation Layer

- Developer designs the UI.
- It enables interaction with user in front-end.

---

3. Tiers

- Application Runs in tiers.
- Web application uses 3 tier architecture.
- Database is hosted and maintained in **"Database Tier"**
- Application is hosted and maintained in **"Application Tier"**
- User uses the application in **"Presentation Tier"**
- The responsibility of technicians in tiers is always about
  a) Maintainance b) Backup
  c) Restoring d) Performance etc.

Back End Layers

a) Database Layer

- MongoDB
- MySQL

b) Business Layer

- Node JS
- Express JS

c) Framework

- MVC
- Microservices

Day 2

Database Layer

1. What is a database?

- It is a store with organized collection.
- Database is a collection with relational data which is good for analytics.

---

2. What are different types of databases?

- **Relational Databases**
  - Good for ERP [Enterprise Resource Planning]
- **No-SQL Databases**
  - Good for ORM [Object Relational Mapping]
- **Hierarchical Databases**
  - Good for network and legacy systems
- **Network Databases**
  - Good for distributed systems
- **Object Oriented Databases**
  - Good for ORM [but have limitations]
- **Distributed Databases**
  - Good for services
- **Cloud Databases**
  - Good in high availability, various geolocation accessibility etc.
- **AI powered Cloud Databases**
  - Good for analytics and reporting

---

3. Database Models

a) Conceptual Model
b) Physical Model
c) Logical Model
d) Entity Model

Day 3

Database Concepts

Topics

- What is Database?
- Types of Databases
- Database Models
    - Conceptual
    - Physical
    - Logical
    - Entity

Conceptual Model

- It represents business requirements.
- It specifies the data to handle.
- It identifies data entities. [modules that require data]
- It configures entity relation but not complete relationship between data.
- It acts are reference for users, analysts and developers.
- It is foundation for “Physical Model”
- It doesn’t contain
  a) Table Names
  b) Field Names
  c) Data Types
  d) Keys etc.

Ex:

- users ⇒ enroll for ⇒ app
    - conceptual model ⇒ user details are stored for further reference
    - ⇒ app needs “users” entity
- app ⇒ uses enrolled data ⇒ for login
    - conceptual model ⇒ user enrolls for login

Pasted Graphic.png ¬

Physical Model

- It represents data requirements.
- It includes table name, field name and data type.
- It comprises of data validations.
- Validation is required to ensure that contradictory and unauthorized data is not get stored into database.
- It defines entity connections with other modules [pseudo entities].

Pasted Graphic 1.png ¬

Logical Model

- It represents data relationships.
- Data required for business and relation with other modules.
- Relation is required for filtering, querying and analysing data.
- It configure primary and foreign keys.
- Primary is a unique identification.
- Foreign is a relation identification.
- It will not define
  a) data type
  b) data validation
- It defines only field name and relation

Pasted Graphic 2.png ¬

Entity Model

- It represent boths business and data requirements.
- It is for users, analysts and developers.
- It comprises of table name, field name, data type, validation and relations.
- It defines entity relations and data relations.
- It is primarily used in ORM. [Object Relational Mapping]
- It enables developers to write for business requirements instead of data requirements.
- Entity configures relations like
  a) One to One
  b) One to Many
  c) Many to Many

unknown.jpg ¬

MongoDB

Day-4

Dec 26, 2025

MongoDB

- It is document oriented database. - It is structure less or semi-structured. - Every record acts as individual document. - It supports data replication.- It

enables high availability and manages load. - It is NoSQL database. - It provides JSON type data, which is BSON. - It provides Javascript like methods for handling interactions. - It support ORM. [Object Relational Mapper] - It supports Ad-Hoc queries.

Concerns - It is not good in handling Relational Data [RDBMS]- It is not good financial and high level ERP. - It is not great for complex analytics.

Setup Environment for MongoDB

- Download and Install MongoDB community server

https://www.mongodb.com/try/download/community

- Select “MongoDB Compass” while installing community server

Server : It is a store where data is kept

Compass : It is GUI client tool.

[latest version provides both CLI & GUI in compass]

- MongoDB server location

mongodb://127.0.0.1:27017 (or) mongodb://localhost:27017

- Open **MongoDB Compass on your PC**

- Select “New Connection”

- Define the connection string “mongodb://127.0.0.1:27017”

- Add & Save

- Connect to MongoDB server using connection.

Note: Windows users make sure that MongoDB server is running.

Programs=> Services.msc => Right Click on “MongoDB Server” =>

start [running]

MongoDB Terminology:

1. Database- It a store where data is kept.- It is a set of collections.

2. Collection- Data in database is stored in the form a collection. - It other databases it is referred as Table.

3. Document- Data in collection is stored as document.- It represents a row or record in table. - It is JSON type. { }

4. Field - Every document has a set of fields.- Field refers to a key in JSON.

5. Embedded Document- It refers to a JOIN in other databases.

MongoDB => Database => Collection => Documents

MySQL / Oracle => Database => Table => Records

Database and Collection

Day-5

Dec 27, 2025

MongoDB Default Databases:

1. admin- It contains admin users. - You create admin users with roles. - You can authenticate and authorize. - Admin users can have access to all databases on your server. - However you can restrict user roles and responsibilities.

2. local- Databases on your server can be replicated to other servers.- So that they are accessible across servers. - “local” is a database where you can configure the collection of settings, which allow to access only on local server.
- Database configured as local can’t replicate to other server.

3. config- It is used to configure the “sharded database”. - Sharded database refers to bulk database. - You can split the database into multiple portions called “Shard”. - Database can split into multiple servers.

MongoDB Custom Database- Database name must start with an alphabet or underscore. - Avoid using other special chars and blank spaces.- Name can be max 64 chars in length. - It can’t be any default database name. - Every database must have a collection. - There is no limit for number of databases on server, it varies according to memory availability. - You can create a new database by using “use” command

```
> use  database_name
```

- In ORM is database_name exists then it start using the database, else it creates the database.

- You can view the list of databases by using

```
> show dbs
```

- You can get active database name by using

```
> db
```

MongoDB Collection- Collection is a location in database, where the data is kept. - It represents a JSON array with collection of objects.

```
[
  { key : value },
  { key : value }
]
```

- Collections in MongoDB are classified into various types.

1. Regular Collection - It stores data normally. - It doesn't have any size limit. - It supports complete CRUD operations. - Documents can be deleted, modified and reordered

2. Capped Collection- Capped is a fixed collection. - It has pre-defined size. - You can store only within the limits. - Older documents are overwritten by new documents. - Insert order is maintained. - Very fast in inserting.

3. Timestamp or Time-Series Collection- It is for time based data. - Time optimized data is kept in time series collection. - It is faster in querying based on time. - It is good for logs. - It is used mostly for monitoring systems.

4. System Collection- It is used for internal storage. - Internal storage refers to meta data. - Meta data is information about your application

5. Views [Readonly Collection] - It is created using aggregation pipeline. - It contains the result of a query and not data. - You can't modify the data. - You can use for querying.

Normal Collection / Regular Collection:

```
> db.createCollection("name")

> show collections
```

Capped Collection

```
> db.createCollection("name", { options } )
```

a) capped:boolean

b) size:number [bytes allocated for collection]

c) max:number [max number of documents]

8bits = 1 byte

1024 bytes =1 kilobytes

1024 kb =1 mega byte

1024 mb = 1 gigabytes

1024 gb = 1 terabyte [tb]

2mb = (1024 x 1024) x 2

2097152

Ex:

> db.createCollection("vendors", {capped:true, size:2097152, max:5})

MongoDB Documents

Day-6

Dec 30, 2025

MongoDB Documents- Collection is a set of documents. - Every document represents an object of Javascript. - Object is used to keep all relative data and logic together. - MongoDB officially uses **BSON** type object. - BSON is **Binary-JSON.** - JSON is string type and BSON is binary type. - BSON have more data types when compared to JSON. - BSON is faster and more efficient than JSON. - It is a Key and value collection. - Key is always string and value can be any BSON type.

```
{

  “key” : value,

}
```

- Document is stored inside collection by using following methods

a) insertOne()

b) insertMany()

Syntax:

db.collectionName.insertOne({ key: value })

db.collectionName.insertMany([{ }, { }])

BSON Types:

1. String - It is literal with group of characters. - String can be enclosed in double or single quotes.

```
{
  "name": "John"  }
```

2. Number

- It can be a signed or unsigned integer.
- **NumberInt()** is used for 32bit integer
- **NumberLong()** is used for 64bit integer
- The default number is 32bit integer

Syntax:

```
{
    age: 20,                                Int32
    price: NumberInt(4000),                 Int32
    mobile:NumberLong(987654321)            Int64
}
```

3. Float / Double / Decimal

- It is used to configure a floating point value.
- It uses decimal places
- BSON defines it as "**Double()**"

- You can configure large decimal values by using “**Decimal128()”**

- Decimal precision can be upto 99.9

Syntax:

```
{
  price: 49.43,
  percent: Double(5993.583),
  sales: Decimal128(998845.3338222)
}
```

- All types prefix must be Number, recommended from latest version

```
{
      NumberDecimal(999884.3949492) // Decimal128()
}
```

- The default number with fraction is considered as double.

4. Boolean- It is used for decision making value. - It handles only “true” and “false” keywords.

Syntax:

```
{
  stock: true
}
```

5. Null - It is used for empty values. - If value is expected at runtime then it is configured as null.

```
{
 rating: null
}
```

6. Date- Date and time can be configured using 2 techniques.

a) Date() constructor to load current date and time.

b) ISODate() to load specific date and time.

- ISODate format must be

“year-month-dayThours:min:secondsZ”

Syntax: Command line methods

```
{
   logTime: new Date(),
   departure: ISODate(‘2025-12-31T15:30:42Z’)
}
```

Syntax: JSON document with date object [GUI]

```
{
   “departure”: {
         “$date”: “2025-12-31T15:40:22Z”
    }
}
```

- Date() constructor allows to configure a new date but not recommend, as it leads to compatibility issues.

Syntax:

```
{  departure : new Date(‘2025-12-30’)   }
```

7. Regular Expression - It is set of meta characters and quantifiers. - It is enclosed in “/ /“.

Syntax:

```
{
```

pattern : /\+91\d{10}/ }

Inserting

Day-6

Dec 30, 2025

MongoDB Documents:

- Collection is a set of documents.
- Every document represents an object of Javascript.
- Object is used to keep all relative data and logic together.
- MongoDB officially uses **BSON** type object.
- BSON is **Binary-JSON.**
- JSON is string type and BSON is binary type.
- BSON have more data types when compared to JSON.
- BSON is faster and more efficient than JSON.
- It is a Key and value collection.
- Key is always string and value can be any BSON type.

  { “key” : value, }

- Document is stored inside collection by using following methods

  a) insertOne()

  b) insertMany()

Syntax:

```
db.collectionName.insertOne( { key: value } )

db.collectionName.insertMany( [ { }, { } ] )
```

BSON Types:

1. String

   - It is literal with group of characters.

   - String can be enclosed in double or single quotes.

```
{
  “name”: “John”
}
```

2. Number

   - It can be a signed or unsigned integer.

   - **NumberInt()** is used for 32bit integer

   - **NumberLong()** is used for 64bit integer

   - The default number is 32bit integer

Syntax:

```
{
    age: 20,                                    Int32
    price: NumberInt(4000),                     Int32
    mobile:NumberLong(987654321)                Int64
}
```

3. Float / Double / Decimal

- It is used to configure a floating point value.
- It uses decimal places
- BSON defines it as “**Double()**”
- You can configure large decimal values by using “**Decimal128()”**
- Decimal precision can be upto 99.9

Syntax:

```
{
   price: 49.43,
   percent: Double(5993.583),
   sales: Decimal128(998845.3338222)
}
```

- All types prefix must be Number, recommended from latest version

```
{
     NumberDecimal(999884.3949492) // Decimal128()
}
```

- The default number with fraction is considered as double.

4. Boolean

- It is used for decision making value.

- It handles only “true” and “false” keywords.

Syntax: { stock: true }

5. Null

- It is used for empty values.

- If value is expected at runtime then it is configured as null.

```
{
 rating: null
}
```

6. Date

- Date and time can be configured using 2 techniques.

  a) Date() constructor to load current date and time.

  b) ISODate() to load specific date and time.

- ISODate format must be

  “year-month-dayThours:min:secondsZ”

Syntax: Command line methods

```
{
```

```
    logTime: new Date(),
    departure: ISODate('2025-12-31T15:30:42Z')
  }
```

Syntax: JSON document with date object [GUI]

```
  {
    "departure": {
        "$date": "2025-12-31T15:40:22Z"
    }
  }
```

- Date() constructor allows to configure a new date but not recommend, as it leads to compatibility issues.

Syntax:

```
  {
    departure : new Date('2025-12-30')
  }
```

7. Regular Expression

- It is set of meta characters and quantifiers.
- It is enclosed in "/ /".

Syntax:

```
{
 pattern : /\+91\d{10}/
}
```

FAQ: How to store date and time values individually?

Ans: MongoDB don’t have any type for separate date and time.

Store only Date:

```
{
  manufactured: ISODate(“2025-12-30”)
}
```

“2025-12-30T0:00:00Z” => split the date or extract date

Store only Time:

```
{
   time: “18:45”          => string type - most recommended
}
```

```
{
    hrs: 18,                    => number type - for manipulation
    min: 45
}
```

8. Array

- It is sequence of elements.
- Array elements can be any BSON type.
- You can configure using meta character “[ ]” or “Array()” constructor.

```
{
    cities: new Array(“A”, “B”, 20, true),
    data : [10, 20, “A”, “B”, false]
}
```

9. Object

- It is key and value collection
- It is considered as embedded object.

```
{
    rating: { rate: 4.5, count: 4500 },
    product: { id:1, name:”TV”, cities: [“Delhi”, “Hyd”] }
}
```

- You can configure a collection of objects

```
{
    data : [ {  }, {  } ]
}
```

10. ObjectId

- It is a unique ID for every document in collection.
- It is a combination of chars and numbers stored in string format.
- The default regular collection generates an “_id” field, which is a unique ID for document with data type as ObjectId.
- You can create a collection without default “_id”.

```
> db.createCollection(“name”, { autoIndexId: false })
```

- MongoDB latest versions have deprecated this property.
- Latest versions using an auto “_id” by default.
- The custom ObjectId must be a 24 chars hexa string.

Syntax:

```
{
    productCode: ObjectId(“hexacode24chars”)                    }
```

11. Binary

- It is used to handle large binary data relative to images, docs etc.
- Storing images, pdf, and other files use binary type.
- However it is not recommended for modern apps.
- Files are always configure with path mapping to any file server.
- Use a string type for files. [path]

FAQ: MongoDB BSON types are strongly typed or implicitly typed?

Ans: All keys configured with data are implicitly typed.

The data type is determined according to the value assigned.

You can store any contradictory data into a field.

FAQ: Can we restrict the types for fields in a document?

Ans : Yes. By using "**Validation Schema".**

Querying Documents from Collection:

- You can query documents by using "**find()"** method.
- It returns one or multiple documents based on a query.
- Query is defined as a "Schema" [object]

Syntax:

```
> db.collection.find({query})
```

- If Query is empty then it returns all documents.

```
> db.products.find({})
```

- The default find() method in latest versions and in compass returns in a pretty format. If you are using a command line for querying documents then can add pretty manually.

```
> db.products.find({}).pretty()
```

Querying documents

Day-7

•

Dec 31, 2025

1. Querying String

- String can be queried using exact match.

```
{field : “value” }
```

- Exact match verifies the capitalization.

```
{ category: “jewelery” }
{ category: “Jewelery” }    // both are different strings.
```

- You can ignore the capitalization and search for string using regular expression with option “i”.

- Regular expression can be used in query by using the following operators.

$regex

$options

- Operators can be used with a subquery.

Syntax:

db.collection.find({ field : { subquery} })

Ex:

db.collection.find({ category: { $regex: “Jewelery”, $options:”i" } })

- String query with regular expression can use all meta characters.

Ex:

db.collection.find({category: {$regex: “Jew*”, $options: “i”}})

- You can query for string inside a sentence at any occurrence without configuring expression in “/ /“.

- “/ /“ verifies the expression with occurrence and quantifier.

Ex:

db.collection.find({title: {$regex: “Sliver*”, $options: “i" }})

Ex:

db.collection.find({category: { $regex: /Jewelery/, $options:”i" }})

2. Querying Number

- You can query for exact matching number.

{ id : 2 }

- Querying numbers for range and condition requires following operators

$gt greater than

$gte greater than or equal to

$lt less than

$lte less than or equal to

$eq equal

$ne not equal

$or logical OR [||]

$and logical AND [&&]

Ex: Price above 600

```
> db.collection.find({ price: {$gte:600 } })
```

Ex: Price between 100 to 600

```
> db.collection.find({$and:[{price:{$gte:100} }, {price:{$lte:600}}]})
```

Ex: All jewelery with price above 600

```
> db.collection.find({$and:[{category:"jewelery"},{price:{$gte:600}]})
```

Ex: All jewelery products and others whose price is above 600.

```
> db.collection.find({$or:[ {category:"jewelery"}, {price:{$gte:600}} ]})
```

Ex: All jewelery and electronics only

```
> db.collection.find({$or:[{category:"jewelery"}, {category:"electronics"}]})
```

3. Querying Boolean

- Boolean is verified by using “true” or “false” keywords.
- Javascript uses 0 or 1 for boolean, which is not recommended in

MongoDB query.

- MongoDB uses 0 or 1 as number type and they are not boolean.

Ex:

db.collection.find({stock:true})

db.collection.find({stock: “true”}) // not boolean

Querying Date, Array, Object

Day-8

•

Jan 2, 2026

Querying Date

- Date and time are configured using ISODate() method.
- You can use the same for querying.
- Date can use operators just like numbers.

$gt

$lt

$gte

$lte etc.

Ex:

```
{
  "Name": "TV",
  "Mfd": {
```

```
    "$date": "2025-12-20T00:00:00.000Z"
  }
}
```

Syntax:

```
db.collection.find({field: ISODate(’2025-12-19’) })
db.demo.find({ Mfd: {$lt: ISODate('2025-12-01')}})
```

Query: Get products manufactured between 10-Nov-2025 and 18-Nov-2025

```
db.collection.find( {$and: [

    {Mfd: {$gte:ISODate(‘2025-11-10’)}},
    { Mfd:{$lte: ISODate(‘2025-11-18’)}}

    ] } )
```

Querying Array

- Array is a collection of values.
- You can query and get values from array using operators like.
  - $in
  - $all
- If you are searching for any one element in array then you can directly query like string.

Syntax:

db.collection.find({ arrayField: “element | value” })

(returns all documents that have specific value in array field)

Ex:

db.demo.find({Topics:"Variables"})

- If you are searching for multiple elements in array then you have to match by using the operators

  $in

  $all

- “$in” is used to find and return documents if any one of the given set of values are found in array.

Ex:

db.demo.find({Topics:{$in:[“Variables","Functions"]}})

(return courses that have either variables or functions)

- “$all” is used to find and return only the documents that have all specified values in array.

Ex:

db.demo.find({Topics:{$all:["Variables","Functions"]}})

(returns only courses that have both variables and functions)

Querying Object

- Object is a key and value collection.
- You can have embedded objects in a document.
- You can invoke the keys with “.” operator.
- The hierarchy of keys are defined as string with “.” operator.

‘object.key.child’

Syntax:

db.collection.find({ ‘parent.child.key’ : value })

db.products.find({‘rating.rate’: 3.3})

Ex: Query all products with rating between 3 and 4

db.products.find({$and:[{ ‘rating.rate’:{$gte:3} }, {‘rating.rate’:{$lte:4} }]})

db.products.find({ ‘rating.rate’: {$gte:3, $lte:4} })

Ex: Query electronics and jewelery products whose rating is between

3 to 4.

db.products.find({category:{$in:["electronics","jewelery"]},'rating.rate':{$gte: 3, $lte:4}})

Aggregation

Day-9

•

Jan 3, 2026

Note: find() returns an array for any query.

You can use “findOne()” that returns only the first occurrence.

find() => [ ]

findOne() => { }

Syntax:

db.products.findOne({id:1})

db.products.findOne({category:’electronics’}) // first electronic product

Displaying on required fields: - find() & findOne() return all fields of collection while displaying query

result.

- You can exclude or include fields to display in result.

1 includes

0 excludes

Syntax:

```
> db.collection.find( { }, { field:0, field:1} )

> db.products.find( {id:2}, {title:1, price:1})   // show only title & price

> db.products.find( {id:2}, {image:0, description:0} ) // show all ignore
                                                        image & description
```

- MongoDB provides “$project” operator to handle include, exclude and compute operations.
- It is mostly used in aggregation. It works like “SELECT” in SQL queries.

```
$project : {
        field : 0,          // exclude field
        field : 1,          // include
field        cal   : expression     // perform specific operation
    }
```

MongoDB Aggregation

- Aggregation is the process of manipulating the data present in collection.
- Data aggregation depends on the type of data in collection.
- MongoDB provides various operators to aggregate different data types.
- Aggregation is an array with multiple expressions.

> db.collection.aggregate([{$project:{ }}])

$project : include, exclude and configure a new field to store the result of any expression.

1. Number Aggregation

- It is the process of manipulating numeric type data, which includes signed, unsigned, float, double, decimal etc.

$sum

$min

$max

$avg

$count

etc.

Query: Find all jewelery products and get the average price of jewellery while price is above 500.

all jewellery products with price above 500

- Aggregate requires a field to configure temporarily so that the result of expression of a specific values is defined into field

dynamically.

Syntax:

```
db.products.aggregate([{$project:{price:1, title:1, avgPrice:{$avg:' '}}}])
```

- Field name is referred as a string with “$”, so that its value is accessed.

Syntax:

```
db.products.aggregate([
    {
        $project:
            {
             price:1,
             title:1,
            avgPrice:{$avg:’$price’}
            }
     }
  ])
```

- To find and return aggregation on specific data you have to filter and group data.
- Aggregation pipeline requires a) $filter b) $groupBy

Note: You can’t directly use any operators in queries while executing from mongoDB Shell, however operators are allowed using script.

Aggregate Functions

Day-10

•

Jan 5, 2026

Aggregation in MongoDB

- Aggregation requires grouping of fields defined using “$group”.
- Aggregation uses “$project” for general operations without group.
- Aggregation uses “$filter” for handling conditions.
- It uses “$match” to find the relative contents while aggregating.

Syntax:

```
> db.collection.aggregate([
        {
            $match: { for filtering data }        // optional
        },
        {
            $group: { },
            $filter: { },
            $project: { }
        }
    ])
```

- $group is an operator used for creating a group and every group requires a unique “_id”.

Syntax:

```
> db.collection.aggregate([
        {
            $group: {
                _id: key_field / null,
                $operator: { expression }  or  field : { expression }
            }
        }
    ])
```

1. Number Aggregation

$sum

$avg

$min

$max

$round

$multiply etc.

Syntax:

```
db.products.aggregate([
        {
            $group: {
                _id: null,
                totalProductCost: { $sum: “$price” }
            }
        }
    ])
```

Syntax:

```
db.products.aggregate([
        {
            $group: {
                _id: null,
                averagePrice: { $avg: “$price” }
            }
        }
    ])
```

Syntax:

```
db.products.aggregate([
        {
            $group: {
```

```
                _id:null,
                leastPrice: { $min: “$price” },
                highestPrice: { $max: “$price” }
            }
        }
    ])
```

Syntax:

```
db.products.aggregate([
    {
        $project : {
            rating: { $round : [ “$rating.rate”, 0 ] }
        }
    }
])
```

- “$rating.rate” object reference for specific key
- “0” refers to number of fractions to round.
- $round doesn’t require group, it can access directly the value and round.
- $group is used on “$sum, $avg, $min, $max”

Syntax:

```
db.fakestore.aggregate([
```

```
        {
            $project: { total: { $multiply: [ “$qty”, “$price” ] } }
        }
    ])
```

2. String Aggregation

$toUpper

$toLower

$concat

$substrCP

$strLenCP etc.

Syntax:

```
db.products.aggregate([
        {
            $project: {
                details: { $concat: [ “$title”,  “belongs to”, “$category” ] }
            }
        }
    ])
```

Syntax:

```
db.employees.aggregrate([
        {
            $project: {
                fullName: { $concat: [ “$FirstName”, “ ”, “$LastName” ] }
              }
        }
])
```

Syntax:

```
db.products.aggregate([
        {
            $project: {
                shortTitle: { $substrCP: [ “$title”, startIndex, endIndex ] }
            }
        }
])
```

Syntax: Find the total length of title

```
db.products.aggregate([
        {
            $project: {
                TitleLength: { $strLenCP: [ “$title” ] }
```

```
            }
        }
    ])
```

3. Date Aggregation

$year

$month

$dayOfMonth etc

Note: All date operations require the type as ISODate()

Syntax:

```
db.products.aggregate([
        {
            $project: {
                ManufacturedMonth : { $month: “$mdf” }
            }
        }
])
```

4. Array Aggregation

$unwind : It is used to destructure the array elements in a field.

It stores array into individual references [spread]

Syntax:

```
db.collection.aggregate([
    {
        $unwind: “$Topics”
    }
])
```

$size : It returns total length of array field.

Update

Day-11

•

Jan 6, 2026

Boolean Aggregation

- It uses true or false as value.
- It is verified using “$match”

```
> db.collection.aggregate([
    {
        $match: {
            inStock: true
        }
    }
])
```

Query: File total amount of all electronic products

```
db.products.aggregate([
        {
            $match: { category: “electronics” }
        },
        {
            $group: {
                _id: “$category”,
                total: { $sum: “$price” }
            }
        }
])
```

Conditions in Aggregation

- Conditional rendering of content is possible in mongoDB
- Condition is configured using “$cond”
- Condition comprises of “if, then and else”.

Syntax:

```
db.collection.aggregate([
    {
```

```
            $project: {
                fieldname: {
                        $cond: {
                            if : { $gt: [ “field”, value ] },     // logical operators
                            then: “ ”,
                            else: “ ”
                        }
                }
            }
        }
 ])


db.products.aggregate([
  {
      $project: {
            Rating: {
                  $cond: {
                        if : { $gte: [ “$rating.rate”, 4 ] },
                        then: “Good Product”,
                        else: “Average Product”
                  }
            }
```

```
        }
    }
])
```

Update Collection

- Update is the process of editing collection, which include modifying data and fields.
- MongoDB provides the methods
    a) updateOne()
    b) updateMany()
- Update requires operators like
    a) $set
    b) $unset
    c) $rename etc.

Updating value in field:

```
db.collection.updateOne(
        {
            find query
        },
        {
            update query
```

```
        }
    )
```

- You can replace value in any field by using “$set”.
- $set allow to update multiple fields. All fields are defined as object with key and value

Syntax: Update one field

```
> db.product.updateOne(
        { id: 1},
        { $set: { price: 302.21 } }
        )
```

Syntax: Update Multiple fields

```
> db.products.updateOne(
        { id: 1},
        { $set: { price: 302.21, category: “electronics” } }
        )
```

Syntax: Updating multiple documents

```
> db.products.updateMany(
```

```
        { category: “electronics” },
        { $set: { “rating.rate”: 3.5 } }
    )
```

Query: Set rating as “4.5” for all products whose price is between 400 to 900.

```
> db.products.updateMany(
        { $and: [{price:{$gte:400}}, {price:{$lte:900}} ] },
        { $set: { “rating.rate”: 4.5 } }
    )
```

Updating values into Array field

1. $push : Adds a new value into array field
   It can duplicate the value.

2. $addToSet : Adds a new value into array but avoids duplication.

3. $each : It is used to add multiple values into array.

4. $position : It is used to add value at specific position.

5. $slice : It can remove values between specific positions.

6. $pop : It can remove both first and last element.

[-1 for first element, 1 for last element]

7. $pull : It is used to remove matching elements from array

Syntax:

```
db.collection.updateOne(
        { find Query },
        { $push: { fieldName: “value” } }
)
```

Syntax: Add multiple

```
{ $push: { fieldName: { $each : [ “val1”, “val2” ] } } }
```

Syntax: Add at specific

```
{ $push: { fieldName: { $each: [ “val1” ] }, $position: 1 } }
```

EX:

```
db.demo.updateOne({Course:"Javascript"},{$push:{Topics:{$each:["Classes","State Management"]}}})
```

Ex:

db.demo.updateOne({Course:"Java"}, { $push: {Topics: "OOPS"} })

Updating Date:

- You can set new data by using ISODate().
- You can’t modify day, month, year individually.

Syntax:

> db.demo.updateOne({id:1}, { $set: { Mfd: ISODate(“your_date”) } })

Add a new field into document(s):

> db.collection.updateMany({ }, { $set:{ newField:value }})

Remove any specific field:

> db.collection.updateMany({ }, { $unset: { fieldName: “ ” } })

Change the field Name

> db.collection.updateMany({ }, { $rename: { “old”, “new” } })

Delete and Schema

Day-12

Jan 7 (Edited Jan 7)

MongoDB Views

- View a virtual collection.
- It comprises of data accessed as a result of any aggregation.
- You can manipulate data in view.
- You can query and use the data.
- Deleting a view will not effect, the actual collection.
- But modification in collection will effect, the view.

Syntax:

```
> db.createView(
        "viewName",
        "collection",            // source data collection
        [
            // aggregation $group, $project
        ]
    )
```

Ex:

```
db.createView(
        "JeweleryCollection",
        "products",
        [
```

```
        { $match: { category: “jewelery” } }
    ]
)
```

Ex: Create a view for electronics with rating above 4

Ex: Create a view with sum of total price for all electronics.

Note: You can view the views as collections.

```
> show collections
```

MongoDB Delete Operations:

- You can remove any document by using

  a) deleteOne()

  b) deleteMany()

- Delete methods use find query.

```
> db.collection.deleteOne({ findQuery })
> db.collection.deleteMany({ findQuery })
```

Ex: Delete all electronic products whose rating is less than 2.

```
> db.collection.deleteMany({ $and:[ { category:"electronics" },
 { "rating.rate": {$lte : 2} } ] })
```

- You can delete any specific field from document by using "$unset", it requires update method.

```
> db.collection.updateOne({findQuery}, { $unset: { } } )
```

- You can delete a collection of view by using "drop()" method

```
> db.collection.drop();
```

- You can delete database by using "dropDatabase()" method.

```
> use database
> db.dropDatabase()
```

Validation Schema for MongoDB

- Validation is the process of verifying user input.
- Validation is required to ensure that contradictory and unauthorized data is not get stored into database.
- MongoDB is not implicitly strongly typed.
- The data type is determined by the value assigned.

- MongoDB provides a validation schema, so that you can set validations explicitly.

Syntax:

```
> db.createCollection("collectionName", { validator: { } })
```

- Validation Schema comprises 3 basic elements

  a) validator

  b) validationLevel

  c) validationAction

- **validator** defines the complete validation schema.
- **validationLevel** defines the restriction which can be "strict" or "moderate".

  a) strict : It will not allows to store if invalid

  b) moderate: It will show warning but allows store.

- **validationAction** defines the error type which can be "error" or "warn".

  a) error : shows invalid

  b) warn : shows as warning

Syntax:

```
> db.createCollection("name",
    {
        validator: { },
        validationLevel: "strict | moderate",
        validationAction: "warn | error"
    }
    )
```

- **Validator** uses a "$jsonSchema" , which is an object that defines

    a) bsonType

    b) required

    c) properties

Syntax:

```
> db.createCollection("name",
    {
        validator: {
            $jsonSchema: {

            },
            validationLevel: "strict",
            validationAction: "error"
        }
    }
```

)

- **$jsonSchema** provides following attributes

  a) bsonType: “object”

  b) required: [“field”, “field”]

  c) properties: { }

- **properties** is an object with collection of keys which refers to fields in collection

```
properties: {
    fieldName: {
        bsonType: “int | string | boolean..”
        required: true,
        minimum: value,
        maximum: value,
        pattern: “regex”,
        description: “error message”
    }
}
```

Ex:

```
db.createCollection("users", {
  validator: {
```

```
    $jsonSchema: {
      bsonType: "object",
      required: ["username", "age"],
      properties: {
        username: {
          bsonType: "string"
        },
        age: {
          bsonType: "int",
          minimum: 15,
          maximum: 30
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})
```

Validations and Atlas

Day-13

•

Jan 8, 2026

Note: MongoDB 4+ version uses “validationLevel” to define “strict” & “moderate”

Syntax:

```
> db.createCollection("users", {

validator: {

  $jsonSchema: {

    bsonType: "object",

    required: ["username", "age"],

    properties: {

      username: {

        bsonType: "string"

      },
```

```
        age: {
          bsonType: “int”,
          minimum: 15,
          maximum: 30
        }
      }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})
```

Validation Properties:

1. String Type

```
minLength        : min length of chars
maxLength        : max length of chars
pattern          : regular expression to match the pattern
enum             : allows only specified values
```

Syntax:

```
properties: {
    city: {
```

```
                bsonType: "string”,
              minLength : 4,
              maxLength: 15,
              enum: [ “Delhi”, “Hyd”]
            }
        }
```

Syntax:

```
    properties: {
            password: {
                bsonType: "string”,
              pattern: “[A-Z]{4,15}”
            }
        }
```

2. Number Type

minumum	: lower bound value
maximum	: upper bound value
exclusiveMinimum	: only greater than specific
exclusiveMaximum	: only less than specific

Syntax:

```
        properties: {
```

```
        age: {
            bsonType: “int”,
            exclusiveMinimum: 15          // above 15 only allowed
        }
    }
```

3. Boolean Type

```
    bsonType : “bool”
```

Syntax:

```
    properties: {
        inStock: {
            bsonType: “bool”,
        }
    }
```

4. Multiple Types

- MongoDB supports union of types.
- It allows a field to handle multiple data types.
- It requires bson type defined with array of types.

Syntax:

```
properties: {
    uname: {
        bsonType: [“string”, “null”],
    }
}
```

5. Date Type

- It is defined using bson type configured as “date”

Syntax:

```
properties: {
    mfd: {
        bsonType: “date”
    }
}
```

6. Array Type

- It uses a bson “array” for collection of data.

Syntax:

```
properties: {
    cities: {
        bsonType: “array”
```

```
        }
    }
```

- You can validate the length of items using

    a) minItems

    b) maxItems

Syntax:

```
    properties: {
        cities: {
            bsonType: “array”,
            minItems: 2,
            maxItems: 5
        }
    }
```

- You can restrict items data type in array by using “items” configured with bson type.

Syntax:

```
    properties: {
        cities: {
            bsonType: “array”,
```

```
            minItems: 2,
            maxItems: 5,
            items: {
                bsonType: “string”  (or)  [“string”, “number”]
            }
        }
    }
```

Note: MongoDB doesn’t provide any specific restriction for object type.

Changing Collection Name

```
> db.collection.renameCollection(“newName”)
```

Note: Changing database name is not allowed. You can use admin commands to create a new database and copy existing into new and remove existing database.

MongoDB Atlas

- It is a cloud server.
- You can host your database on cloud to integrate with apps on cloud.
- Login into MongoDB Atlas

  https://cloud.mongodb.com/ - Configure a “free” account
- Create a cluster on free account.

- Setup username & password [generates connection string]

- You can create a database & collection

MySQL

Day-14

•

Jan 9

FAQ: What is B-Tree?

Ans: It allows to configure a collection to make querying faster than normal.

It is the process of configuring indexes in a collection.

FAQ: What is the purpose of Indexes?

Ans: It makes querying fast by selecting exactly the matching content.

Without index query will scan every document in collection.

With index it directly access the required document.

```
> db.collection.createIndex( { field : 1 / -1 })
```

1 = ascending order

-1 = descending order

Every collection has a default index by name “_id”.

Ex:

```
> db.products.createIndex({id:1})
```

```
> db.products.find({id:4})              // makes query faster
```

- Indexes are of various types

  a) Single Index

  b) Compound Index

  c) Sparse Index

  d) Unique Index

  e) Multi Key Index [Array]

- You can check the status by using “explain()” of mongoDB

```
> db.collection.find({ }).explain(“executionStatus”)
```

- You can remove index dropIndex()
- You can view index by using getIndexs()

MySQL Database

- It is an open source, cross platform, free database from Oracle.
  * Open source allows to customize.
  * Cross platform allows to run across various OS services.
  * Free License
- It is a Relational Database Management System.
- It is SQL based database.

MySQL Architecture

1. **MySQL Connectors** - Responsible for enabling connection with Application

- Application can be .NET, ODBC, JDBS, Node.js, Python etc.
- Connectors use providers and drivers to connect with application.

2. **MySQL Shell**

- It provides a command line interface [CLI] for interacting with database.
- You can handle interactions with commands.

3. **MySQL Server Process**

- It provides NoSQL and SQL interfaces.
- NoSQL interface is responsible for handling CSV type data.
- SQL interface handles DDL, DML etc.
- Responsible for querying and CRUD operations.

4. Storage Engines

- InnoDB : for transactions [PK, FK default]
- MyISAM : faster in reading (no transactions)
- Memory : temporary data
- NDB Cluster : for shared data

5. File System

- It is the location where data is stored.

- It comprises files, logs, data etc.

Key Features of MySQL:

- Open Source & Free

- Cross platform

- Fast and scalable

- ACID features with InnoDB

- Secure (users, roles, prevelizations)

Setup & Install MySQL on your PC

1. Download Community Server

   https://dev.mysql.com/downloads/mysql/

2. Install Community Server on your device

   a) provide a password for “root” user.

3. Download and Install My SQL Workbench

https://dev.mysql.com/downloads/workbench/

4. Open MySQL Workbench create a connection with server.

RDBMS

Day-15

•

Jan 10, 2026

RDBMS

- Relational Database Management System
- It refers to data configured with relationship.
- Relational Database stores data in tables.
- Table comprises of relations with
    a) rows (records / tuples)
    b) columns (fields / attributes)
- Relational Data contains tables connected with keys.
- The key features of RDBMS
    a) Data stored in tables
    b) Predefined Schema
    c) Relationship using keys
    d) ACID compliance
    e) Uses SQL
    f) Strong Data Integrity

What is ACID?

- ACID Refers to

A : Atmocity

C : Consistency

I : Isolation

D : Durability

- Atmocity specifies all or nothing. It returns the specified result or will not return anything.

There is no suspense.

- Consistency refers to a validation state.

If valid it performs the given functionality.

If not valid the reject the action.

- Isolation is the process of handling concurrent queries

It is the process of executing your queries without effecting

other queries in process.

- Durability is committing the actions defined.

RDBMS vs NoSQL

Structured	Semi Structured or Structure Less
Relation with Keys	No Relation

Vertically Scaled	Horizontally Scaled
ACID Compliance	Partially ACID
Use SQL language	No SQL

MySQL Objects

Day-16

•

Jan 19,2026

MySQL Objects

1. Database

   - It is a store used to handle data on server.

   - Data is kept in database.

2. Table

   - It comprises of data.

   - Data is arranged into rows and columns.

   - Table contains structured data.

3. Record

   - It refers to a row.

   - Row refers to a “tuple” that contains various types of value.

4. Field

   - It refers to a column header in table.

   - Data in a row can be accessed with reference of field name.

5. View

- It is a virtual table.
- It comprises of temporary data.
- Data summarized using queries can be stored in a view.
- You can’t modify data is view, but can use the data from view.

6. Stored Procedure

- Procedure is used to “Refactor” the code.
- Refactor is the process of extracting a set of statements into a block.
- It enables easy reusability and extensibility.

7. Function

- A function is used to build expression.
- Expression executes and evaluates a value.
- It always returns a value.

8. Trigger

- A trigger is like an event which is used to execute function and procedure.
- Triggers are used in designing batch operations.
- Batch is a sequence of actions performed with one trigger.

Note: Database can also have various other objects like forms, reports, macros, BI and Analytics etc.

SQL

- SQL is "**Structured Query Language".**

- It is a language used to handle communication with database in **RDBMS.**

- Developers communicate with database using **SQL** as language.

- SQL provides a set of commands which are categorized into

a) DDL

b) DML

c) DCL

- **DDL is Data Definition Language**, which provides commands for handling data definition like creating database, views, procedures, triggers etc.

a) Defining Database

b) Defining View

c) Defining Procedure etc.

- **DML is Data Manipulation Language,** which provides commands for handling interactions with database, procedures, triggers etc.

a) Filtering Data

b) Querying Data

c) Updating Data

d) Deleting etc.

- **DCL is Data Control Language,** which provides commands to handle authorization and authentication.

  a) Authorization is the process of restricting access to specific object.

  b) Authentication is the process of verifying credentials.

Data Definition Language

- DDL is about configuring database, procedures, views, triggers etc.
- "**create"** is a command used for creating various objects.
- "**use"** is a command for using object.

Database

- It is a store on server.
- In RDBMS database is a "**Schema".**
- Schema comprises of structured data.
- Structured data is good for analytics and reporting.
- You can create a new database by using 2 techniques

> create database name

> create schema name

- The default database is “**sys**” which refers to database server system.
- It comprises of tables, views, procedures and functions for managing server system.

FAQ: What is character set for database? Why we need Character Set?

Ans: Database can handle various typed of data.

Character set allows to configure complex data types.

It controls the language, storage and case sensitivity.

Charset allows emojis, multilingual content etc.

The charset used for emoji and multilingual is “**utf8mb4**” and it must collate with “**utf8mb4_unicode_ci**”.

Ex:

```
create database projectdb
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

MySQL Data Types and Constraints

Day-17

•

20 Jan, 2026

DDL

Creating Database

Character Set “utf8mb4”

Collate “utf8mb4_unicode_ci”

Use Database

- You can handle interactions with any database only when it is active.

- You can make any database active by using “use” command.

```
> use database_name ;
```

Creating Tables

- Table arranges data in rows and columns.

- It provides a schema for data.

- Schema refers to specified structure for data.

Syntax:

```
CREATE  TABLE   table_name
(
    field_name    data_type    [constraints],
    field_name    data_type    [constraints]
)
```

- Field Name refers to column “header”, It is a key to access value in table.

- Data Type defines the data structure, the type & range of data that can be stored in a field.

- Constraints are used to configure rules for fields.

MySQL Data Types

1. Numeric Data Types

TINYINT	very small integer	Age / Boolean 0 or 1
SMALLINT	small integer	Day number, year number etc.
MEDIUMINT	medium integer	Grade / Scores without decimal

INT / INTEGER standard integer (Numeric) Sales / Cost without decimal

BIGINT very large integer Student Id, Employee ID etc.

FLOAT Small Fractions Interest Rate

DOUBLE Medium Fractions Salary , Price etc.

DECIMAL Large Fractions Price, Sales with fractions.

BOOLEAN Bool [TINYINT] 0 or 1 Stock, Active etc.

2. String Data Types

CHAR(n) Fixed length string

VARCHAR(n) Variable Character Length

TINYTEXT Small Text

TEXT Medium Text

MEDIUMTEXT Large Text

LONGTEXT Very Large Text

BINARY(n) Fixed binary

VARBINARY(n) Variable Binary

ENUM Enumeration with set of constant values , Fixed Set

SET Variable with multiple sets of values.

3. Date Type

DATE Handles a Date Value

TIME Handles Time value

DATETIME Handles both date and time

TIMESTAMP Auto Updated Date and Time

YEAR Handles Year Value

4. Binary & Large Object

BLOB	Binary Large Object	Documents
TINYBLOB		
MEDIUMBLOB		
LONGBLOB		

5. JSON Type

- It is new from MySQL 5+ versions
- JSON is Javascript Object Notation
- It is a key and value collection.

JSON	Key / Value collection

MySQL Constraints

- Constraints define rules for fields in a table.
- Rules about the behaviour of data in fields.
- Constraints can verify data across rows in a table.
- The constraints are categorized into various groups

1. Column Constraints

NOT NULL

UNIQUE

DEFAULT

AUTO_INCREMENT

PRIMARY KEY

CHECK

COMMENT

2. Foreign Key Constraints

FOREIGN KEY

CASCADE

SET NULL

RESTRICT

NO ACTION

PRIMARY KEY CONSTRAINT

- RDBMS requires a unique identification for every record in database table.

- It is used for indexing.

- Indexing is required for handle interactions like update, delete etc.

- Primary Key can't be null. The field configured as primary key must have a value. - Primary Key must be unique. Value can't be same across records.

Syntax:

```
CREATE TABLE  products
(
    ProductId TINYINT PRIMARY KEY,
    Name VARCHAR(20),
    Price DECIMAL(10,2)
)
```

NOT NULL CONSTRAINT

- It specifies that field must have a value.

- It is used for mandatory fields.

- Field is considered as "Required" field.

Syntax:

```
CREATE TABLE products
(
  ProductId  TINYINT PRIMARY KEY,
  Name VARCHAR(20) NOT NULL
)
```

UNIQUE CONSTRAINT

- It specifies that the value can’t be same across records.
- Field value must be unique.
- You can’t use duplicate value.

Syntax:

```
CREATE TABLE  products
(
  ProductId TINYINT PRIMARY KEY,
  Name VARCHAR(20) NOT NULL,
  ProductCode VARCHAR(10) NOT NULL  UNIQUE
)
```

Constraints

Day-18

•

21 Jan, 2026

Summary:

- **Creating Table**
- **MySQL Data Types**

a) Number

b) String

c) Date

d) Binary

e) JSON

- Constraints

a) Column Constraints

b) Foreign Key Constraint

- **Column Constraints**

a) Primary Key

b) Unique

c) Not Null

Not Null Constraint

- Every field in table is by default nullable.

- You can save record without a value inside field.

- **NOT NULL** is used for required fields.

- If any field is set with Not Null then it must have a value.

Syntax:

```
CREATE TABLE  products
(
    ProductId  TINYINT  PRIMARY KEY,          Name VARCHAR(20) NOT NULL
```

)

DEFAULT CONSTRAINT

- If a field is not defined with value then it can use any default value.

- You can configure a default constraint.

- Default can be any MySQL type.

- Default value is overwritten with custom value.

- If value is provided while inserting document then the default is replaced with custom

value.

Syntax:

```
CREATE TABLE  products
(
    ProductId  TINYINT  PRIMARY KEY,          Name VARCHAR(20) NOT NULL,
    Qty INT  DEFAULT 1
)
```

CHECK CONSTRAINT

- A field usually allows any value of specified type.

- User can input any value.

- Check constraint is used to verify the input value.

- It can check the value using various conditions and store only when matching with given condition.

- The operators required for building check expression

=	Equal	< >	Not Equal
>	Greater than		
>=	Greater than or equal to		
<	Lessthan		
<=	Lessthan or equal to		
AND	Logical &&		
OR	Logical ||		
NOT	Logical !		

Syntax:

```
CREATE TABLE  products
(
   ProductId  TINYINT  PRIMARY KEY,          Name VARCHAR(20) NOT NULL,
   Qty  INT  DEFAULT 1  CHECK (Qty<=3)
)
```

AUTO_INCREMENT CONSTRAINT

- You can configure a field with numeric type data.
- The value in field can auto increment based on the previous.
- Incremented value is assigned to next record.
- The default incrementation starts with ‘1’.

- It can increase the value by 1.

- It uses “ENUM” behaviour for values in a field.

Syntax:

```
CREATE TABLE  products

(

   ProductId  TINYINT  PRIMARY KEY AUTO_INCREMENT,          Name
VARCHAR(20) NOT NULL

)
```

Note: If there is no previous value, then it starts with “1”.

If there is a previous value, then it increments the previous value with 1.

FOREIGN KEY CONSTRAINT

- It is used to configure relationship between tables.

- A field can be absolute or relative.

- Absolute fields are not connected with others. They are individual. - Relative fields are connected with others.

- A relation can be

  a) One to One

  b) One to Many

  c) Many to Many etc.

- Foreign key constraint is used to connect a table field with another table.

Syntax:

```
Create Table  tableName
(
    field_name  data_type,
    constraints  constraint_name   FOREIGN KEY(fieldName)
    REFERENCES  tableName(fieldName)
    ON UPDATE  “Mode”,
    ON DELETE  “Mode”
)
```

- Constraint name is required to modify or delete constraints.

- **Foreign Key** refers to field name of current table, which you to use as foreign key.

- **References** are used to configure the table that contains foreign key field.

- Mode can be any one of the following

    a) STRICT

    b) CASCADE

    c) SET NULL

- The default mode is “Strict”.

- If update and delete are in strict mode then it will not allow to modify or delete

  the foreign key field value.

Syntax:

```
CREATE TABLE  categories
(
   CategoryId  TINYINT PRIMARY KEY  AUTO_INCREMENT,
   CategoryName VARCHAR(20)
)

CREATE TABLE products
(
   ProductId  TINYINT PRIMARY KEY,
   Name VARCHAR(20),                    CategoryId TINYINT,
   CONSTRAINTS  fk_category   FOREIGN KEY(CategoryId)
   REFERENCES   categories(CategoryId)
   ON DELETE STRICT
   ON UPDATE STRICT
)
```

Relationship

Day-19

•

Jan 22, 2026

Foreign Key Constraint Modes

1. **Restrict**
2. **Cascade**
3. **Set Null**

- The default is “Restrict”, it will not allow to delete or update the value in field if it is already in use.

- Cascade will remove all relative records when key field is delete.

  Cascade will update all relative records with new value if modified.

- Set Null will keep “null” as value if relative data is deleted or modified.

What is a Relation in Database?

- Relation defines how data of one table is connected with data in another table.
- Relation is required for Data Integrity, Data Analysis, Data Prediction etc.
- Relations are categorized into following types

1. One-to-One
2. One-to-Many
3. Many-to-One
4. Many-to-Many
5. Self Reference Relation

Demo Project : I-Shop [IEEE]

- It is an e-commerce application.
- Online Shopping
- Modules

    1. Registration Module
    2. Products Browse
    3. Products Search
    4. Shopping Cart
    5. Shipping & Billing Module
    6. Payment Module

7. Admin User Management

8. Admin Catalog Management

9. Admin Order Management

Data Models:

1. Conceptual Model

   - User-Register

   - Products

   - Categories

   - Shopping-Cart

   - Shipping

   - Admin

2. Physical Model

   tblUsers

UserId	VARCHAR(20)
UserName	VARCHAR(20)
Password	VARCHAR(20)
Email	VARCHAR(20)
Address	VARCHAR(100)
Mobile	VARCHAR(20)

   tblProducts

ProductId	TINYINT
Name	VARCHAR(20)
Price	DECIMAL(8,2)
Qty	TINYINT
Image	VARCHAR(50)

CategoryId TINYINT

tblCategories

CategoryId TINYINT

Name VARCHAR(20)

tblCart

CartId TINYINT

UserId VARCHAR(20)

ProductId TINYINT

tblShipping

VendorID TINYINT

Name VARCHAR(20)

Address VARCHAR(100)

CategoryId TINYINT

tblAdmin

AdminId VARCHAR(20)

Name VARCHAR(20)

Password VARCHAR(20)

3. Logical Model

Insert, Select QUERY

Day-20

Jan 23, 2026

2. Physical Model

tblUsers

UserId	VARCHAR(20)
UserName	VARCHAR(20)
Password	VARCHAR(20)
Email	VARCHAR(20)
Address	VARCHAR(100)
Mobile	VARCHAR(20)

tblProducts

ProductId	TINYINT
Name	VARCHAR(20)
Price	DECIMAL(8,2)
Qty	TINYINT

Image VARCHAR(50)

CategoryId TINYINT

tblCategories

CategoryId TINYINT

Name VARCHAR(20)

tblCart

CartId TINYINT

UserId VARCHAR(20)

ProductId TINYINT

tblShipping

VendorID TINYINT

Name VARCHAR(20)

Address VARCHAR(100)

CategoryId TINYINT

tblAdmin

AdminId VARCHAR(20)

Name VARCHAR(20)

Password VARCHAR(20)

3. Logical Model

tblUsers

UserId [PK]

UserName

Password

Email

Address

tblCategories

CategoryId [PK]

Name

tblProducts

ProductId [PK]

Name

Price

Image

Qty

CategoryId [FK]

tblCart

CartId [PK]

UserId [FK]

ProductId [FK]

tblShipping

VendorId [PK]

Name

Address

CategoryId [FK]

tblAdmin

AdminId [PK]

Name

Password

Syntax: Configuring Multiple Foreign Keys

```
create table tblCart
(
 CartId TINYINT PRIMARY KEY,
 UserId VARCHAR(20),
 ProductId TINYINT,
 constraint fk_user FOREIGN KEY(UserId) REFERENCES tblUsers(UserId),
 constraint fk_product FOREIGN KEY(ProductId) REFERENCES tblProducts(ProductId)
)
```

Summary

- Creating Table
- MySQL Types
- Constraints
- Relationship

Relationships in I-Shop:

1. One to One
   - Every User Can have one Shopping Cart
   - Every Shopping Cart can be only for one user.

2. One to Many

- One vendor can supply products of multiple categories.
- One category can have multiple products.

3. Many to One

- Many products can refers to one category.
- Many categories can refers to one vendor.

4. Many to Many

- Many products can be in many categories.
- Many vendors can refer to many categories.

Inserting Data into Tables

- SQL provides a Data Manipulation Language command "**insert",** which is used to insert data into a table.

Syntax:

```
INSERT INTO TableName (Field1, Field2) VALUES (Value1, Value2);
```

- Field Name refers to the column header in table.
- Value must match exactly with the field data type.
- Numeric values are directly defined. [20, 45000.33, 2.4]
- String values are defined using single quote. ['John']
- Date values are defined using single quote, but must use the format 'yy-mm-dd'.
- You can insert multiple records by using "," delimiter.

Syntax:

```
INSERT INTO table (field1, field2) VALUES (va1, val2), (val1, val2) ;
```

Ex:

```
Insert into tblUsers (UserId, UserName) values ('john_nit', 'John'), ('sam', 'samson');
```

```
INSERT INTO tblProducts (ProductId, Name, Mfd) VALUES (1, 'TV', '2026-01-10')
```

Querying Data from Table

- SQL provides **SELECT** command to query and read data from a table.
- SELECT can use wild card or field names to access data from table.

Syntax:

```
SELECT * FROM table_name;
SELECT Field1, Field2 FROM table_name;
```

Note: The wild card "*" is not recommended at production level.

- SELECT query can have the following

a) Wild Card

b) Field Name

c) Alias Name

d) Expression

Syntax:

SELECT Field1 as Alias FROM table_name;

Note: Alias is a friendly reference name given for field to display in query result.

Ex:

SELECT ProductId AS ‘Product Code’, Name AS ‘Product Name’ FROM tblProducts;

Syntax:

SELECT Field AS Alias, (expression) AS Alias from table_Name;

Note: Expression refers to operations on data fields.

Expression can use various operators numeric, string etc.

Ex:

SELECT Name AS 'Product Name', Price, Qty AS 'Quantity', (Qty * Price) AS 'Total' from tblProducts;

Select QUERY

Day-21

Jan 24, 2026

Summary

- Insert

- Select

a) Field

b) Wild Card

c) Expression

d) Alias

WHERE COMMAND

- It is used to filter the data.

- It can sort, group or aggregate data using various operators combination.

- The query operators for numeric, string and date types are:

=	Equal
< >	Not Equal
>	Greater than
>=	Greater than or Equal
<	Less than
<=	Lessthan or equal
AND	Logical AND (&&)
OR	Logical OR (||)
NOT	Logical NOT (!)

Syntax:

```
SELECT  Fields  FROM  table  WHERE  (expression);
```

Queries:

1. Select product with ID 2.

```
SELECT  *  FROM  tblProducts  WHERE   ProductId=2;
```

2. Select products manufactured on 10 January 2026.

```
SELECT * FROM  tblProducts  WHERE  Mfd='2026-01-10';
```

3. Product with price above 20000

SELECT * FROM tblProducts WHERE Price>=20000;

4. Select all Electronic products with Price above 20000

SELECT * FROM tblProduct WHERE Category='Electronics' AND Price>=20000;

- SQL provides "**LIMIT"** command to limit the number of records displayed as result.

SELECT * FROM table WHERE (expression) LIMIT number;

Ex:

SELECT * FROM tblProducts WHERE Price>20000 LIMIT 2;

[returns only top 2 results]

SELECT * FROM tblProducts WHERE Price>20000 LIMIT 2,4;

[returns the 3rd result]

SELECT * FROM tblProducts WHERE Price>20000 LIMIT 2,8;

[returns from 2nd to 8th]

- SQL provides "**BETWEEN"** operator to find the values in specified range.

Syntax:

SELECT * FROM tblProducts WHERE Field BETWEEN (LowerBound AND UpperBound)

Lower Bound = Minimum

Upper Bound = Maximum

Ex: Select products with price between 10000 and 20000.

```
SELECT * FROM tblProducts WHERE  Price BETWEEN 10000 AND 20000;
SELECT * FROM tblProducts WHERE  Price > 10000 AND Price < 20000;
```

- SQL provides **"IN"** operator to find the values that match exactly the given set.

Syntax:

```
SELECT  * FROM table  WHERE   Field  IN (Value1, Value2,..);
```

Ex: Select and return products that belong to electronics & jewelery

```
SELECT * FROM tblProducts WHERE  Category IN ('Electronics', 'Jewelery');
```

Ex: Select electronic & jewelery products with price range between 10000 to 20000

```
SELECT * FROM tblProduct WHERE Category IN ('Electronics', 'Jewelery') AND
Price BETWEEN (10000 AND 20000)
```

- SQL provides "**LIKE"** operator to find a value inside given string type data. It uses the following wild cards

a) %

b) _

- The wild card “%” is used to find at any occurrence of a string. It refers to one or multiple characters.

Syntax:

SELECT * FROM tblProducts WHERE Category=‘Jewelery’ AND Name LIKE ‘%Gold%’;

- The wild card “_” is used to find a string matching with specified character at specified position. It refers to any single character.

Syntax:

SELECT * FROM tblProducts WHERE Name Like ‘_at’; bat, mat,

Ex: Select all Jewelery and Electronic products where name have “Silver” word.

- SQL provides operators for finding the records with Null.

  a) IS NULL

  b) IS NOT NULL

Syntax:

SELECT * FROM table WHERE Field IS NULL;

SELECT * FROM table WHERE Field IS NOT NULL;

Ex: Select all products that don’t have a rating value.

SELECT * FROM tblProducts WHERE Rating IS NULL;

Ex: Select all electronic & jewelery products with price range between 10000 to 20000 and have rating.

- SQL provides **DISTINCT** to avoid duplicates.

Syntax:

```
SELECT DISTINCT FROM Table;
```

- SQL provides **ORDER BY** command for sorting data. It uses “ASC & DESC”

Syntax:

```
SELECT * FROM table ORDER BY Field ASC / DESC;
```

Ex: Select top 2 products with highest rating.

```
SELECT * FROM tblProducts ORDER BY Rating DESC  LIMIT 2;
```

Group & Aggregation

SQL Functions

Day-22

Jan 26, 2026

Summary

- SELECT
- WHERE
- IN
- BETWEEN
- LIMIT

- LIKE

- ORDER BY

- IS NULL

- IS NOT NULL

- GROUP BY

- DISTINCT

MySQL Functions

- Function is used to refactor the code and return a value.

- MySQL provides several pre-defined functions to manipulate the data in fields.

- Functions must always return a value using expression.

- SQL functions can be categorized into various groups based on the type of value they return.

String Functions

1. ASCII

   - It is used to return the ASCII code of specified character.

   - ASCII uses the code 65 to 90 for Uppercase & 97 to 122 for Lowercase.

Syntax:

```
SELECT    ASCII('A');

SELECT  FieldName, ASCII(FieldName)  from TableName;
```

   - If you are referring to field name then it returns the ASCII of first character.

Ex:

```
SELECT   Name,  ASCII(Name)  from  tblProducts;
```

2. CHAR_LENGTH

- It returns the total count of characters in a string.

- You can refers to field to get the length of chars for data in field.

Syntax:

```
SELECT   CHAR_LENGTH('Your Text');

SELECT    FieldName, CHAR_LENGTH(FieldName) from Table;
```

Ex:

```
select Name, char_length(Name) from tblProducts;
```

3. SUBSTRING

- It returns the specified numbers of chars from given index.

- It requires start index and count of chars.

Syntax:

```
SELECT    SUBSTRING( 'your string', start_index, count_of_chars);

SELECT   Field,  SUBSTRING( Field, 1, 10 );
```

1= First Index

10 = Upto 10 Chars [10th included]

- Reference index always starts with "1" for string chars.

Ex:

```
select Name, substring(Name, 1, 3) from tblProducts;
```

4. SUBSTR

- It is a short hand method of SUBSTRING.
- Both are same.

Syntax:

```
SELECT    SUBSTR('Your Text', start_index, count_of_chars);
```

5. TRIM

- It is used to return a string by removing the leading spaces.
- Leading space refers to space before and after string.

Syntax:

```
SELECT    TRIM(' your text ') ;
SELECT   Field,  TRIM (Field) from Table;
```

Ex:

```
Select TRIM('  Welcome ') AS 'RESULT';
```

6. CONCAT

- It is used to concatenate multiple strings and return as one string.

Syntax:

```
SELECT  CONCACT( 'String1' , 'String2', …) ;
```

Ex:

Select FirstName, LastName, CONCAT(FirstName, LastName) AS ‘Full Name’ from tblEmployees;

SELECT UserId, UserName, concat(UserId, ‘-‘ , UserName) as 'User Details' from tblUsers;

7. UPPER

- It converts the chars into uppercase letters.

8. LOWER

- It converts the chars into lowercase letters.

Syntax:

SELECT UPPER(‘Your string’);

Select Field, Upper(Field) from table;

Ex: Write query to return ASCII code of first character in a field. It must return only first char.

SELECT substring(Name,1,1) , ASCII(substring(Name,1,1)) from tblProducts;

Number Functions

- SQL Provides various functions to handle operations and formats on numeric data.

- It includes parsing, numeric evaluation, round, aggregate etc.

- Number functions include

1. ABS()

2. ROUND()

3. FLOOR()

4. CEIL()

5. MIN()

6. MAX()

7. AVG()

8. COUNT() etc.

Syntax:

```
SELECT COUNT(*) from table;        // returns count of all records
SELECT COUNT(*) from table where (expression);
SELECT SUM(Field) AS 'ALIAS';
SELECT Field, SUM(Field) GROUP BY Field;
```

Ex:

```
SELECT Category, Count(*) from tblProducts Group By Category;
```

Number Functions Samples

Day-23

Jan 26, 2026

MySQL Number Functions Exercises with Solutions

Sample Data Table: products

```
CREATE TABLE products (
   id INT,
   name VARCHAR(50),
   price DECIMAL(10, 2),
   quantity INT
);
```

```
INSERT INTO products VALUES
(1, 'Laptop', 59999.99, 10),
(2, 'Mouse', 499.50, 100),
(3, 'Keyboard', 1049.75, 50),
(4, 'Monitor', 12000.40, 20),
(5, 'Webcam', 2999.99, 15);
```

1. Retrieve product names and absolute difference between price and 5000

```
SELECT name, ABS(price - 5000) AS price_difference FROM products;
```

2. Show product names with prices rounded to the nearest whole number

```
SELECT name, ROUND(price) AS rounded_price FROM products;
```

3. Show prices rounded to 1 decimal place

```
SELECT name, ROUND(price, 1) AS rounded_price FROM products;
```

4. Display product names with ceiling value of price

```
SELECT name, CEIL(price) AS ceiling_price FROM products;
```

5. Display product names with floor value of price

```
SELECT name, FLOOR(price) AS floor_price FROM products;
```

6. Truncate price values to 2 decimal places

SELECT name, TRUNCATE(price, 2) AS truncated_price FROM products;

7. Find remainder when quantity is divided by 7

SELECT name, MOD(quantity, 7) AS remainder FROM products;

8. Display square of each product's quantity

SELECT name, POWER(quantity, 2) AS quantity_square FROM products;

9. Find square root of price for each product

SELECT name, SQRT(price) AS price_sqrt FROM products;

10. Show product names and if price is above 10000 using SIGN()

SELECT name, SIGN(price - 10000) AS price_sign FROM products;

11. Display least of price and quantity for each product

SELECT name, LEAST(price, quantity) AS least_value FROM products;

12. Display greatest of price and quantity for each product

SELECT name, GREATEST(price, quantity) AS greatest_value FROM products;

13. Show total stock value of each product (price × quantity), rounded to 2 decimal places

SELECT name, ROUND(price * quantity, 2) AS total_value FROM products;

14. Find which products have a price % 5 = 0

SELECT name FROM products WHERE MOD(price, 5) = 0;

15. Calculate POWER(price, 1/2) and compare it with SQRT(price)

SELECT name, POWER(price, 0.5) AS price_pow_sqrt,
SQRT(price) AS price_sqrt FROM products;

16. Which products have a floor value of price above 10000

SELECT name FROM products WHERE FLOOR(price) > 10000;

17. List products whose ABS(price - quantity) is less than 1000

SELECT name FROM products WHERE ABS(price - quantity) <
1000;

18. Divide quantity by 3, rounded to 1 decimal place

SELECT name, ROUND(quantity / 3, 1) AS divided_quantity FROM
products;

19. Show all products with random value assigned using RAND()

SELECT name, RAND() AS random_value FROM products;

20. Add a new column 'adjusted_price' = CEIL(price * 1.05)

ALTER TABLE products ADD COLUMN adjusted_price DECIMAL(10,
2);

UPDATE products SET adjusted_price = CEIL(price * 1.05);

Date and Time functions

Day-24

Jan 27, 2026

Summary

- **String Functions**

- Number Functions

MySQL Date Functions

- MySQL provides following data types for storing date and time values.

a) DATE

b) TIME

c) DATETIME

Syntax:

```
Create Table Name
(
  Field   DATE,
  Field   TIME,
  Field   DATETIME
)
```

- Data is inserted into date and time fields using following formats

a) DATE “year-month-day”

b) TIME “hours:minutes:seconds.microseconds”

c) DATETIME “yy-mm-dd hrs:min:sec.micro”

- MySQL provides various date and time functions for querying and manipulating date and time type of data.

1. CURRENT_DATE

- It returns the current date value.
- Date value includes Year-Month-Day
- The alias for the function is “curdate()”

2. CURRENT_TIME

- It returns the current time value.
- Time value includes Hours, Minutes, Seconds, Micro Seconds.
- The alias is “curtime()”

Note: The date and time functions can be used as a getter or setter.

As they get and set current date and time.

Syntax:

```
Create Table userlogtime
(
    logid tinyint primary key auto_increment,
    logtime time
)

Insert Into userlogtime  (logtime)  values (curtime());
```

Query: Write a query to get the total logtime of all users.

3. NOW()

- It returns the current date and time.

- It returns “yy-mm-dd hrs:min:sec”

Syntax:

```
SELECT  NOW();
```

4. WEEKDAY()

- It returns the weekday number.

- Weekday number starts with “0=monday”.

- It requires a date value to return weekday from specified date.

Syntax:

```
SELECT   WEEKDAY('2026-01-26');
SELECT   WEEKDAY(curdate());
```

5. WEEK()

- It returns the current week number, in current year.

Syntax:

```
SELECT WEEK()
```

6. DAYNAME()

- It can return the weekday name from specified date.

Syntax:

```
SELECT  DAYNAME(curdate());
```

7. MONTH()

- It returns the month number.

- Month number starts with “1 = January”.

Syntax:

SELECT MONTH(curdate());

8. MONTHNAME()

- It returns the month name form specified date.

Syntax:

SELECT MONTHNAME(curdate());

Query: Return all products manufactured month name, which are manufactured after March.

Table:

Id,

Name

Price

Mfd [Date]

Name	Manufactured Month
TV	April
Mobile	June

Select Name, monthname(Mfd) as ‘Manufactured Month’ from tblproducts

Where month(mfd) > 3;

Query : Return all electronic category products manufactured between jan-april.

Select * from tblproducts where category=‘electronics’ AND month(mfd) between (1 and 4);

9. DATE()

10. TIME()

11. HOURS()

12. MINUTES()

13. SECONDS()

14. MICROSECONDS()

15. YEAR()

Summary

- **String**
- **Number**
- **Date & Time**

Decision Making, Conversion, Admin

Day-25

•

Jan 28, 2026

Summary

- Number Functions

- String Functions

- Date Functions

MySQL Conversion Functions

1. CAST()

- It is a SQL function used to convert a value into specific data type.
- CAST is a temporary function, which will not change the data type permanently.

Syntax:

```
SELECT   CAST ( Data  AS  Type );
```

Data refers to the value in a field.

Type refers to any MySQL data type.

Ex:

```
SELECT CAST('2026-01-22' AS DATE) as ‘Manufactured’;
```

2. CONVERT()

- It is a SQL function used to convert one type into another permanently.
- It can save the data into specified type.

Syntax:

SELECT CONVERT(data, type);

data : It refers to the value.

type : It refers to any SQL data type.

Ex:

SELECT CONVERT('2026-01-22', DATE) as 'Manufactured'

MySQL Decision Making Functions:

1. IF( )

- It is a decision making function.
- It uses a boolean value or expression.
- It executes one set of actions when expression evaluates to **true,** and another set of actions when it evaluates to **false.**

Syntax:

SELECT Field, IF(BooleanExpression, Action_on_true, Action_on_false) AS 'Alias' from tableName;

Ex:

```
create table products
(
  Id TINYINT primary key auto_increment,
  Name varchar(20),
  Rating INT
)
```

Insert Into products (Name, Rating) values ('TV', 4), ('Mobile', 2), ('Watch', 5),

('Shirt', 3), ('Jeans', 3)

Syntax:

SELECT Name, Rating, IF(Rating>=4, 'Good Product - You can buy', 'Average Product - Often returned') AS 'Product Reviews' from products;

- You can use all SQL operators for building a boolean expression.

- You can use all SQL commands for filtering and sorting data while using condition.

Ex: Same query but display only reviews for electronic and jewelery products.

SELECT Name, Rating, IF(Rating>=4, 'Good Product - You can buy', 'Average Product - Often returned') AS 'Product Reviews' from products where Category IN

(‘Electronics’, ‘Jewelery’);

Ex: Same query but ignore all products that don’t have rating.

SELECT Name, Rating, IF(Rating>=4, 'Good Product - You can buy', 'Average Product - Often returned') AS 'Product Reviews' from products where Rating IS NOT NULL;

2. CASE

- It is a switch like selector used to select any one from a given set of conditions.

- It is used for selecting a block of actions to perform, when there are multiple choices.

Syntax:

```
SELECT Field,
CASE
    WHEN  Condition  THEN  Action_to_Perform
    WHEN    Condition    THEN  Action_to_Perform
END
AS
'Alias_name' from TableName;
```

Ex: Select name and rating, if rating is < 3 then poor, if rating is 4 then average, if rating is > 4 then Good

```
SELECT Name, Rating,
    CASE
        WHEN Rating<=3 THEN 'Poor'
        WHEN Rating=4 THEN 'Average'
        WHEN Rating>4 THEN 'Good'
    END
AS 'Product Reviews' from products where Rating IS NOT NULL;
```

Ex:

Customer Table

- Name
- City

Product Table

- Name
- Price
- Vendor

- VendorCity

Vendor Table

- Name
- City

IF Customer City is equal to Vendor City then “Same day Delivery”

Else “Delivery by Tomorrow”

Ex:

```
create table Cart
(
  Id TINYINT primary key auto_increment,
  UserName varchar(20),
  City varchar(20)
 )

insert into Cart (UserName, City) values ('John', 'Delhi'), ('David', 'Hyd'),
('Sam', 'Chennai')

SELECT UserName, City,
CASE
WHEN City='Delhi' THEN 'Delivery in 2 days'
WHEN City='Chennai' THEN 'Delivery by Tomorrow'
WHEN City='Hyd' THEN 'Same day delivery'
END
AS 'Delivery Time' from Cart
```

MySQL Admin Functions:

1. USER()

- It returns the current user of database terminal on server.

2. DATABASE()

- It returns the database details.

3. VERSION()

- It returns the SQL version details.

Syntax:

```
SELECT  USER();

SELECT DATABASE()
```

MySQL Update Query

Update and ALTER

Day-26

Jan 29, 2026

SQL DML Command

1. UPDATE

- It is used to update the value of any field in database table.
- It can set a new value by replacing the existing.
- It uses a helper command **SET.**
- It requires UPDATE to find a relative record and make changes.
- To find any record UPDATE can use “**WHERE**” helper.

Syntax:

UPDATE TABLE SET Field=Value Where (Condition);

- The value must match with the data type of field.
- The default WHERE can use on reference of any PRIMARY KEY field.

Syntax:

UPDATE products SET Rating=4 where Id=2;

UPDATE products SET Rating=4 where Name='Mobile'; // invalid

- The default MySQL SAFE UPDATES are ON [1]. You have update only by using primary key field.

- You can turn OFF safe update and enable updating of values by any field.

Syntax:

SET SQL_SAFE_UPDATES = 0; // OFF

SET SQL_SAFE_UPDATES = 1; // ON

Ex: SET rating 5 for all products above 40000.

UPDATE product SET Rating=5 WHERE Price>=40000;

- You can update multiple fields using UPDATE query.

Syntax:

UPDATE products SET Name='TV', Price=42000, where Id=1;

2. ALTER

- It is used to modify the table structure.
- Modifying table structure includes
  a) Adding Fields
  b) Changing Field Name
  c) Removing Fields
  d) Adding, Modifying and Removing Constraints etc.
- It uses helper methods for changing the structure, which include
  a) ADD
  b) MODIFY
  c) CHANGE
  d) DROP etc.

Manipulations: [ALTER]

1. Adding a new field into table

- ALTER command can use ADD helper to add a new field into table.
- Every new field must be configured with Data Type and Constraints.
- You can set position for new field by using the helpers **AFTER & FIRST.**

Syntax:

ALTER TABLE table_name ADD new_field data_type constraints FIRST / AFTER existing_field_name;

- Existing field name is required only when you place AFTER.

Note: You can get table structure using “desc” [describe] command.

```
desc table_name;
```

Syntax:

```
ALTER TABLE products ADD Price Decimal(8,2) NOT NULL AFTER Name;
ALTER TABLE products ADD Price Decimal(8,2) NOT NULL  FIRST;
```

2. Change the structure of any existing Field

- You can modify the field structure by using **MODIFY** helper.
- Field structure includes data type and constraint, not name.
- You can’t modify the structure of field if there is data in field.
- You can’t modify the structure if field is used as foreign key and with RESTRICT mode.

Syntax:

```
ALTER TABLE table_name MODIFY  field_name  data_type constraint;
```

Ex:

```
ALTER TABLE products MODIFY Price Decimal(8,2) AFTER Name;
```

3. Change the field name

- You can change the table field name by using **CHANGE COLUMN** helpers.
- It requires the old and new names.
- You can’t change the name of fields that are configured as foreign keys with RESTRICT mode.
- You can set a new position for field while changing name by using AFTER & FIRST.

Syntax:

ALTER TABLE table_name CHANGE COLUMN old_field_name new_field_name AFTER existing_field_name

- The name must be configured with structure. It can have data type and constraints.

Syntax:

ALTER TABLE table_name CHANGE COLUMN old new data_Type constraint;

Ex:

ALTER TABLE products CHANGE COLUMN Name Title varchar(20) AFTER Id;

4. Removing Field from table

- ALTER can remove any field by using **DROP COLUMN** helper.

- You can drop the column only when it is not configured as foreign key with RESTRICT.

Syntax:

ALTER TABLE table_name DROP COLUMN field_name;

- You can remove the field with data. Make sure that data is not connected to foreign key.

Ex:

ALTER TABLE products DROP COLUMN Price;

- You can use DROP to drop the primary key.

- It will remove primary from table.

- You can remove primary key and field by using field name.

- You can't remove primary key if is defined as "auto_increment".

- You can't remove if it is set as foreign key.

Syntax:

```
ALTER   TABLE   tableName DROP  PRIMARY KEY;      // removes primary key constraint.
```

DELETE & Stored Procedures

Day-27

Jan 31, 2026

DELETE COMMAND

- It is used to remove records from database table.

- It can find the records using WHERE helper.

- It can limit the records to delete using **LIMIT** helper.

Syntax:

```
DELETE  FROM  table_name   WHERE   (Condition);

DELETE  FROM  products WHERE   Id=1;

DELETE  FROM  products WHERE  Rating<=3  LIMIT 2;
```

- You can't delete a record used as foreign key with RESTRICT.

Summary

- CREATE

- SELECT

- ALTER

- UPDATE

- DELETE

Stored Procedures

- A Stored procedure is used to **Refactor** the code.
- Refactor is the process of extracting a set of statements and enclosing in block. So that you can reuse without rewriting.
- It is easy to reuse, extend and maintain.
- Programming languages can refactor code using 3 techniques
  a) Function
  b) Method
  c) Procedure
- Function is intended to return a value.
- Method is void type.
- Procedure can return or can be void, it can change according to the state & situation.

MySQL Procedure:

- A procedure is configured with a **DELIMITER**
- DELIMITER is used a separator in query.
- Multiple queries can be separated using DELIMITER
- The default DELIMITER in MySQL is **"$$".**

Syntax:

```
DELIMITER  $$

    … your stored procedure …

DELIMITER
```

- Make sure that the procedure is inside DELIMITER scope.

- Procedure is created by using **CREATE PROCEDURE** command.

```
CREATE  PROCEDURE  Name(Params)
   BEGIN


   END$$
```

- Every procedure has 3 phases

a) Declaration

b) Signature

c) Definition

- Declaration is used to configure a procedure

Syntax:

Create Procedure Name(params)

- Signature is used to call the procedure from any location.

Syntax:

Call Name(params)

- Definition defines the actions to perform. It comprises of statements to execute.

Syntax:

BEGIN

```
… your statements …
END
```

Syntax:

```
DELIMITER $$

    Create Procedure Name()
    BEGIN
        Statements;
    END

DELIMITER ;
```

Ex: Create a procedure to get top rated products

```
DELIMITER $$
  CREATE PROCEDURE GetTopProducts()
      BEGIN
        SELECT * FROM products where Rating>=4;
      END$$
DELIMITER ;
```

Call GetTopProducts() => to execute procedure

Parameters:

- A procedure can be parameter less or parameterized.

- Parameter less procedure defines the same functionality across multiple requests.

- Parameter is required to modify the functionality.

- Parameters are configured as

    a) Formal

    b) Actual

- The parameters defined in declaration are formal parameters

Create Procedure Name(formalParameter)

- The parameters defined while calling procedure are actual parameters.

Call Name(actualParameter)

formalParameter = actualParameter;

- Parameters in a stored procedure are categorized into 3 types

    a) IN

    b) OUT

    c) INOUT

- The following keywords are defined for formal parameters

    a) IN

    b) OUT

    c) INOUT

- Every formal parameter requires a name and its data type.

- Parameter data type can by any SQL type.

Syntax:

```
Create Procedure Name(IN Param Data_Type)
BEGIN
    … you can use the parameter …
END
```

- **IN** parameter is used to configure parameter as a **CONSTANT.**
- It keeps the value constant across the statements.
- Every parameter is a required parameter and have order dependency.

Ex:

```
DELIMITER $$
    CREATE PROCEDURE GetByRating(IN Rate TINYINT)
    BEGIN
      SELECT * FROM products where Rating=Rate;
   END$$
DELIMITER ;
```

Parameters and Variables

Day-28

•

Jan 31, 2026

Summary

- **Stored Procedure**
- **DELIMITER**

- Parameters

- IN Parameter

OUT Parameter:

- It is used to render the result of any expression in a procedure.
- It will not allow to define any value explicitly.
- It can store a value that is returned as a result of any expression.
- It will not allow to change the value explicitly but can change implicitly according to state and situation.

Syntax:

```
create procedure Name( OUT  param_name  dataType )
    BEGIN
        …
    END
```

- Statements can store the result of any expression inside out parameter by using **"INTO"** command.

- Expression can store the result into OUT parameter by using **SET** command.

Syntax:

```
BEGIN
    Select  avg(Price) INTO out_param_name FROM tableName;

    … you can use the out_param in various statements …
```

```
END
```

- You can call the stored procedure with out parameters by using the parameter reference defined with “@“.

Syntax:

```
call   Name();         // invalid parameter missing
call   Name(20);       // invalid out parameter will not accept a value
call   Name(@out_param_name);
```

- You can render the result of OUT parameter by using SELECT command.

Syntax:

```
SELECT   @out_param_name;
```

Ex: Create a procedure with OUT parameter that store the average price of all products.

```
DELIMITER $$
  Create procedure GetAvgPrice(OUT avgPrice decimal(8,2))
   BEGIN
     SELECT AVG(Price) INTO avgPrice from tblProducts;
   END$$
DELIMITER ;
```

Query: Create a procedure with IN & OUT params where we can pass the category name and it returns the avg price of the category related products.

[category name is IN param, avgPrice is OUT param]

```
call  Name('electronics', @avgPrice);
```

O/P:

'Average of Electronics'

34000.442

Syntax:

```
create procedure GetAvgByCategory(IN category varchar(20), OUT avgPrice decimal(8,2))
BEGIN
        SELECT  AVG(Price) INTO avgPrice from tblProducts where Category=category;
END
```

```
call  GetAvgByCategory( 'Electronics', @avgPrice )
SELECT   @avgPrice AS 'Average of Electronics';
```

- If you have to handle multiple values dynamically in a stored procedure then you can configure variables instead of defining all as OUT parameters.

Variables in Stored Procedure:

- Variables are storage locations in memory where you can store a value and use it as a part of any expression.
- Variable configuration comprises of 3 parts

a) Declaring

b) Assignment

c) Initialization

- SQL variables are declared by using **DECLARE** command

Syntax:

```
DECLARE  variable_name  data_type;
```

- You can assign value into variable using **SET** command

Syntax:

```
SET  variable_name = value;
```

- A procedure can store value into variable using **INTO** command, if it is in a query.

Syntax:

```
SELECT   function()  INTO  variableName from tableName;
```

- You can initialize value into variable by using **DEFAULT** command.

Syntax:

```
DECLARE  variable_name  data_type  DEFAULT  value;
```

Ex:

```
DELIMITER $$
```

```
Create procedure GetAvgPrice(OUT avgPrice decimal(8,2))

BEGIN


   DECLARE  a_price decimal(8,2) DEFAULT 0.00;


    SELECT AVG(Price) INTO a_price from tblProducts;


    SET avgPrice = a_price;


END$$

DELIMITER ;
```

JOINS

Day-29

4 Feb, 2026

INOUT Parameter:

- It can configure a value dynamically into reference and return the value.
- It has both IN and OUT behaviour.
- However value is provided as input explicitly it is provided as result of expression or from any field.

Syntax:

```
DELIMITER $$

Create procedure GetOfferPrice(INOUT offer_price decimal(8,2), IN offer decimal(8,2), IN Id TINYINT)

BEGIN


    SELECT Price INTO offer_price from tblProducts where ProductId=Id;


     SET offer_price = offer_price-offer;
```

```
END$$
DELIMITER ;

call GetOfferPrice(@offer_price, 200, 2);
select @offer_price AS 'Final Offer Price';
```

JOINS in MySQL

- Joins are used to compare data in two different tables.
- Join can verify data from multiple tables and return the similarities and differences.
- Joins are categorized into different types

a) Inner Join

b) Left Join

c) Right Join

d) Cross Join

- Join is created by using SELECT statement, which selects data from two tables.

a) Source is known as “Left Table”

b) Target is known as “Right Table” [target refers to the table to be compared with]

Syntax:

SELECT all_fields_from_both_tables AS alias_name FROM left_table JOIN_TYPE right_table ON query/condition.

- INNER JOIN

- It compares data in two tables and returns only the matching records.
- It ignores the other records.
- It will not display mismatched data in both tables.

- LEFT JOIN

- It returns all records of left table.
- It compares with right table.
- It sets null for the records in right table where matching data not found.

- RIGHT JOIN

- It returns only the matching data of both tables.
- It gets all right table fields in order.
- It returns the left table field in order of right table.

- CROSS JOIN

- It ignores the records that are not matching in both tables.
- LEFT & RIGHT joins will return null if different data is found.
- INNER & CROSS join will ignore such data.

Syntax:

```
SELECT  all_vendors.VendorId, all_vendors.VendorName AS 'All Vendors',
            hyd_vendors.VendorId, hyd_vendors.VendorName AS 'Hyd Vendors'
  FROM      all_vendors INNER JOIN hyd_vendors
ON         all_vendors.VendorName=hyd_vendors.VendorName;
```

Tables:

all_vendors

VendorId	VendorName
1	Bajaj
2	Reliance
3	Sony

hyd_vendors

1	Sony
2	Bajaj
3	LG

MySQL Views

- View is a virtual table.
- You can query data from tables and store in a view.
- It can used in queries for sorting, filtering like actions.
- It will not allow to modify the data.
- It is automatically updated according to the modifications in source table

Syntax:

```
CREATE VIEW  View_name  AS  query.
```

- Query is a SELECT query with various commands and operators.

Query: Create a view for storing all high rated products [rating >= 4]

```
CREATE VIEW  top_rated_products  AS  Select * from tblProducts Where Rating>=4;
```

- View data is accessed with SELECT query.

```
SELECT  * from View_Name;
```

Triggers

Triggers

Day-30

5 Feb, 2026

MySQL Triggers

- Trigger is a database object which is used to execute [trigger] a block of code when ever specific action is performed on database table.

- Trigger executes implicitly for various actions in data table.
- Trigger has events like

  a) INSERT

  b) UPDATE

  c) DELETE

- Trigger has timings like

  a) BEFORE

  b) AFTER

- INSERT defines adding of a new record into table.

- UPDATE defines changes in any field of a record in table.

- DELETE defines removing of any record from table.

- BEFORE specifies that trigger executes before the Event.

- AFTER specifies that trigger executes after the event.

Syntax:

```
DELIMITER $$

    Create Trigger  trigger_name
    TIMING  EVENT  ON  table_name
    FOR EACH ROW

    BEGIN

        // defines actions to be performed by trigger

    END $$

DELIMITER ;
```

- The table fields are accessed inside trigger by using “NEW” operator.

```
    BEGIN
```

```
        NEW.Field_Name;


    END $$
```

Ex:

1. Create a new database table

```
    create table  products_data
    (
      id  tinyint  primary key  auto_increment,
      name varchar(20),                price decimal(8,2),                created_at
datetime,
      modified_at datetime
    )
```

2. Add a new trigger for capturing create date time for every record when inserted into table

```
    DELIMITER $$


     Create Trigger insert_product_trigger
      BEFORE INSERT ON products_data
     FOR EACH ROW
        BEGIN
          SET NEW.created_at = now();
       END $$
```

```
DELIMITER ;
```

3. Try inserting a record

```
INSERT Into products_data (name, price) values ('tv' , 34000.44)
```

4. You can use "SIGNAL SQLSTATE 45000" to enable custom errors and display while updating, inserting & deleting.

Query: Create a trigger to verify price and make sure that it will not insert a negative value.

```
DELIMITER $$

Create Trigger  validate_product_price
BEFORE  INSERT  ON products_data
FOR EACH ROW

  BEGIN
      IF   NEW.price <=0  THEN
         SIGNAL  SQLSTATE  '45000'
         SET NEW_MESSAGE = 'Price must be above 0'
      END IF
  END $$

DELIMITER
```

Note: SIGNAL is to indicate SQL language

SQLSTATE is to access and verify the current state

45000 is error code number, which is used for custom errors.

NEW_MESSAGE / MESSAGE_TEXT for displaying error message.

Query: Create a trigger to capture the update time for every record when updated.

5. You can see all triggers of a database by using

“Show Triggers”

6. You can remove trigger by using “Drop”

Drop Trigger trigger_name;